Welcome the class and name the question for today: when does a tidy tree help, and when does it hide a relationship? Tell students that every example uses synthetic data only.

Connect to Week 12 in one minute. A procedure can be perfectly executed and still answer the wrong question if its model is wrong. Today the trap is a change in edge meaning: dependency is a work-order relationship; containment is a displayed-under relationship. Ask for a prediction before showing the graph.

Preview the sequence and the loop. Students should predict before code runs, then compare the visible result, then explain the gap. Do not start with abstract definitions; the project picture comes next. The answer to the prediction is breadth-first, but do not give it away yet.

Do not read every row aloud. Point out that graph, directed edge, tree, and traversal are the words that will recur. Emphasize the two meanings: dependency tells us about completion; containment tells us about display. If a term is new later, return to this slide.

Keep the example concrete. Read the seven labels and say that they are invented for the lesson. Ask which items project points to directly. Students should predict plan and build. This is a dependency question, not yet a folder question.

Define the edge in plain language and give three examples. Pause on requirements to tests: it will cross branches later. Ask students to say the sentence with the arrow reversed; reversing it changes the claim. The common mistake is the exact model drift we are auditing.

Draw attention to the red dashed cross-link. The six nearby edges show a natural hierarchy, but requirements to tests is a separate dependency. Have students name the seven edges: two from project, two from plan, two from build, and the cross-link.

Make the distinction explicit before converting the graph. The dependency graph says requirements must precede tests, but it does not say where a directory tree must draw tests. Ask the prediction; the answer is no. A hierarchy can choose a convenient display without preserving every dependency edge.

Say that we are making a useful display, not claiming to replace the graph. Freeze the child order now because every traversal answer depends on it. Ask the class to predict the missing edge before showing the tree.

Walk the tree from the root and read the children left to right. Count seven vertices and six tree edges. Say explicitly that a tree with seven vertices has six parent-child edges, so it cannot carry all seven graph dependencies without an extra relation. That count is a clue, not the whole audit.

Use the table as a side-by-side translation. Students often assume a tree edge keeps the old meaning; it does not unless we say so. Ask which row predicts a lost cross-branch relation. The answer is the third row.

Point at the six solid tree edges, then the red dashed edge. The hierarchy is not bad; it is incomplete for the dependency question. Name the exact dropped edge. This is the lesson's central audit result and will be generated and tested from the reference code.

Turn the picture into a small, falsifiable check. The set subtraction is more reliable than trusting a tidy drawing. Ask what should happen to the stronger claim; the test should fail or be marked as an expected failure. We will keep that failure visible later.

Define each traversal with a concrete sentence. The child order is part of the input, not a cosmetic choice. Ask the prediction: breadth-first reaches build before requirements because both are children of project.

Trace with a finger: project, enter plan, enter requirements, then design, then return and enter build. Say that the printed sequence is generated from code/trees.py, so the slide cannot quietly drift from the tested answer.

Walk the plan branch first: requirements, design, plan. Then build: tests, docs, build. Only then project. Ask which vertex must be last; the root. Explain that postorder often matches “finish parts before the whole,” but it still follows containment edges, not every dependency.

Describe the queue in plain language: put project in, remove it, add plan and build, then remove plan before build. This is why the two top-level children appear before their children. The exact sequence comes from the code-derived macro.

Show the error as intentional. Ask students what check exposes it: inspect levels, or run the breadth-first test. The wrong trace dives down the plan branch before visiting build, so it is depth-first preorder. Fix it by using the queue-based function and the exact generated trace.

Compare the three sequences without asking which is best. Each is a correct answer to a different traversal rule. Close the section by saying that tests freeze the exact order, which is why a plausible-looking trace is not enough.

Introduce the supplied Tuesday scenario as another concrete tree. Point out every yes/no label on an arrow. The labels describe which branch was taken; the nodes are Start, questions, and outcomes. This is the cleaned decision tree required for the package.

Read the route slowly and ask students to point to the selected arrows. The trace does not include “yes” as a node. Make the distinction concrete: the route records visited nodes, while the arrow labels explain each choice.

Compare the decision-tree traces with the project-tree traces. The code uses the same traversal functions, but the child order is different: needs aid? comes before explain rule. Ask students to look for yes/no in the lists; they should not find it.

State the limitation without inventing a new policy. The displayed tree ends at aid route, while the intake note adds a separate relation to language-access review. The tree can be useful and still incomplete. Ask what a stronger conclusion would wrongly assume.

Have students compare the bounded and unbounded claims. The first is directly checked by a trace. The second needs information not represented in the tree. Do not rewrite the Decision Gate or turn the omission into a real-person example; keep it synthetic and model-focused.

Give about three minutes. Students predict the seven vertices and edges, then run prompt-a. Listen for the phrase “A must be completed before B.” The model comes before the procedure.

Students run prompt-b after prompt-a. The exact-set test is the visible check; do not allow later code to silently change the data.

Give about four minutes. Have students convert the graph into the fixed directory-style tree and name the edge meaning before they judge the drawing.

Students run the structural checks: root, one parent, seven nodes, and acyclicity. The test checks the tree shape, not whether it preserves every dependency.

Give about four minutes. Students implement preorder, postorder, and breadth-first traversal. Remind them that the fixed child order is part of the input.

Students compare the exact assertions. If breadth-first looks like preorder, ask where the queue is. Inorder is not part of this general rooted-tree lesson.

Give about four minutes. Students model the eligible-plus-aid route, keeping yes/no as edge labels. No real intake data is needed or allowed.

Students test the three decision traces and identify what the tree does not show: aid route to language-access review.

Give about four minutes. Students compare graph edges with tree edges and predict the one missing relationship before running the audit.

Students add the regression test and keep the strict expected failure. Explain that an expected failure is evidence about the false claim, not a broken suite.

Give about four minutes. Students search the seven synthetic tree nodes and include the missing language-access review case.

Students compare node coverage with the dropped-edge audit. The intended conclusion is no: finding nodes does not prove that all dependency edges were preserved.

Give about five minutes. Students run the full suite and print both project and decision traces plus the dropped set. Require the bounded sentence from the prompt and label direct checks versus model limitations.

Use this as the final visual audit. The first six happen to line up with the chosen hierarchy, but the seventh does not. Say “happen to” because the meanings differ; a matching shape does not prove identical semantics.

Read the conclusion once and ask students to underline its limit. The first half is supported by the tests; the second half is the model boundary exposed by the audit. This is the one-sentence style to carry into the Decision Gate.

Spend about three minutes previewing the first half of the five headings: Sources, Rules/Assumptions, and the start of Work. Tell students their artifact needs a bounded claim and one omitted relation. The text is included byte-for-byte from the repo Decision Gate.

Finish the preview with Work, Check Your Answer, and One-Sentence Summary. The exact submission location and due time are deliberately not inferred from this source package.

Bridge without turning Thursday into divided labor. Each student should reason about the claim, then compare interpretations with a partner. The output is a bounded Pair Reasoning report, not a duplicate of the Decision Gate. Carry forward the named missing edge and the habit of checking relationships, not only nodes.

Close in one minute. Ask students to say what an edge means before they choose a traversal, and what relationship the tree leaves out before they make a strong claim. The answer for this lesson is requirements to tests; the language-access relation is the corresponding decision-tree limitation.

Use this to find a slide fast: slide number, lesson idea, and prompt file. Prompt a is the first block and b is the second block.

Use this to find a slide fast: slide number, lesson idea, and prompt file. Prompt a is the first block and b is the second block.

Use this to find a slide fast: slide number, lesson idea, and prompt file. Prompt a is the first block and b is the second block.

Use this to find a slide fast: slide number, lesson idea, and prompt file. Prompt a is the first block and b is the second block.

Use this to find a slide fast: slide number, lesson idea, and prompt file. Prompt a is the first block and b is the second block.